

productRouter.js



Giải thích Router `productRouter.js`

Router có vai trò là **người điều phối**. Nó nhận một yêu cầu HTTP (GET, POST, v.v.) từ client, khớp nó với một đường dẫn (path) cụ thể, và chuyển giao việc xử lý cho một hàm Controller tương ứng.

1. Khai báo Modules

```
const express = require("express")
const upload = require("../middlewares/uploadFiles");

const {
  getAllProduct,
  getFormProduct,
  createProduct,
  deleteProduct
} = require("../controllers/productController");
```

- `const express = require("express")` : Tải framework Express.
- `const upload = require("../middlewares/uploadFiles");` : Tải **middleware xử lý upload file** (rất có thể là đã cấu hình `multer`) từ thư mục `middlewares`. Middleware này sẽ được chèn vào các route cần tải tệp tin.
- `const { ... } = require("../controllers/productController");` : Tải các **hàm xử lý** (Action) từ `productController.js` (mà bạn đã gửi trước đó). Sử dụng cú pháp **Destructuring** (`{ }`) để lấy ra trực tiếp các hàm cần thiết.

2. Khởi tạo Router

```
const productRouter = express.Router()
```

- `express.Router()` : Tạo một **đối tượng Router** mới. Đối tượng này cho phép bạn định nghĩa các route một cách độc lập và sau đó gắn chúng vào ứng dụng chính (`app.js`) bằng `app.use()`.

3. Định tuyến và Gắn Controller

Tất cả các route dưới đây đều sử dụng đối tượng `productRouter`.

A. Hiển thị Danh sách (READ)

```
//display list
productRouter.get("/", getAllProduct)
```

- `productRouter.get("/")` : Khớp với yêu cầu **GET** đến đường dẫn cơ sở của Router này.
 - Ví dụ: Nếu trong `app.js` bạn gắn Router này là `app.use("/products", productRouter)` , thì route này tương ứng với URL: `/products` .
- Nó sẽ thực thi hàm `getAllProduct` để lấy và hiển thị danh sách sản phẩm.

B. Hiển thị Form Thêm (GET Form)

```
//form create new
productRouter.get("/add", getFormProduct)
```

- `productRouter.get("/add")` : Khớp với yêu cầu **GET** đến đường dẫn `/add` .
 - Ví dụ: Tương ứng với URL: `/products/add` .
- Nó thực thi hàm `getFormProduct` để kết xuất giao diện form thêm sản phẩm.

C. Xử lý Thêm Sản Phẩm (CREATE)

```
//control create new
productRouter.post("/add",upload.single("image"), createProduct);
```

- `productRouter.post("/add")` : Khớp với yêu cầu **POST** (dữ liệu form được gửi đi) đến đường dẫn `/add` .
- `upload.single("image")` : Đây là **Middleware xử lý File Upload**.
 - Nó được chèn vào **giữa** đường dẫn và Controller.
 - `upload` là đối tượng Multer đã được cấu hình.
 - `.single("image")` chỉ ra rằng chỉ có một tệp tin được upload, và tên trường (field name) trong form HTML phải là `image` .
 - *Cách vận hành*: Khi yêu cầu POST đến, middleware này sẽ chạy trước, xử lý file, lưu file vào thư mục quy định, và gắn thông tin file đã xử lý vào `req.file` . Sau đó, nó chuyển quyền điều khiển sang hàm `createProduct` .
- Nó thực thi hàm `createProduct` để lưu dữ liệu và tên file vào CSDL.

D. Xóa Sản Phẩm (DELETE)

```
//delete
productRouter.get("/delete/:_id", deleteProduct)
```

- `productRouter.get("/delete/:_id")` : Khớp với yêu cầu **GET** đến đường dẫn `/delete/` kèm theo một tham số.
 - `:` (dấu hai chấm) đánh dấu `_id` là một **tham số động** (URL Parameter).
 - Ví dụ: Tương ứng với URL như `/products/delete/65f4d8e8a...` .
- Giá trị của tham số này (ID) sẽ được lấy ra trong Controller bằng `req.params._id` .
- Nó thực thi hàm `deleteProduct` để tìm và xóa sản phẩm tương ứng.

4. Xuất Module

```
module.exports = productRouter;
```

- Xuất đối tượng `productRouter` để file `app.js` có thể `require()` và gắn nó vào base path `/products` (hoặc bất kỳ base path nào khác).

Router này là chuẩn mực trong việc tách biệt các mối quan tâm (separation of concerns).